



TRIGGERS

en PostgreSQL

Con ejemplos de sistemas en producción real

Base de Datos Avanzadas

UP Chiapas · Mtro. Ramsés Camas



github.com/RamsesCamas/BDA_Slides_And_Code

Evaluación del Curso

50%

Defensa oral individual

Demostrar competencia en vivo: escribir y explicar código SQL/PL sin ayuda

25%

Proyecto final con defensa

Proyecto integrador con preguntas individuales a cada miembro del equipo

15%

Quizzes en clase (pizarrón/oral)

Preguntas al azar al inicio de clase — sin dispositivos, sin aviso previo

10%

Labs entregados

Ejercicios prácticos de laboratorio entregados a tiempo



Mínimo 60 en defensa oral o se reprueba automáticamente, sin importar los demás componentes.

Agenda de Hoy



¿Qué es un Trigger? Anatomía y sintaxis

20 min



¿Cuándo SÍ y cuándo NO usar Triggers?

10 min



5 ejemplos en producción real (10 → 10K usuarios)

40 min



Buenas prácticas y anti-patrones

15 min



Ejercicio en clase con Docker

15 min

Escala de ejemplos:

10 users

40 users

100 users

1K users

10K users



Requisito: Docker Compose del repositorio levantado + PL/pgSQL de la clase pasada.

¿Qué es un Trigger?

Código que se ejecuta automáticamente ante INSERT, UPDATE o DELETE en una tabla.



Analogía: Es como una alarma — no la activas manualmente, se dispara sola cuando alguien abre la puerta.



Se asocia a una tabla específica



Responde a eventos DML (INSERT, UPDATE, DELETE)



Se ejecuta ANTES o DESPUÉS del evento



Es una función PL/pgSQL que retorna TRIGGER



Un trigger NO puede hacer COMMIT ni ROLLBACK — vive dentro de la transacción que lo disparó.

Anatomía de un Trigger

PostgreSQL separa el trigger en 2 partes obligatorias:

1 Trigger Function (QUÉ hacer)

```
CREATE OR REPLACE FUNCTION fn_audit()  
RETURNS TRIGGER AS $$  
BEGIN  
  
  -- Tu lógica aquí  
  
  RETURN NEW;  
END; $$ LANGUAGE plpgsql;
```

BEFORE vs AFTER:

	BEFORE	AFTER
Timing	Antes del cambio	Después del cambio
¿Modifica datos?	Sí (cambiar NEW)	No
¿Cancela op?	Sí (RETURN NULL)	No
Uso típico	Validar, autocompletar	Auditoría, notificar

2 CREATE TRIGGER (CUÁNDO disparar)

```
CREATE TRIGGER trg_audit_emp  
AFTER INSERT OR UPDATE OR DELETE  
ON empleados  
FOR EACH ROW  
EXECUTE FUNCTION fn_audit();
```

Filtro opcional con WHEN:

```
WHEN (NEW.precio != OLD.precio)
```

OLD y NEW por evento:

Evento	OLD	NEW
INSERT	✗	✓
UPDATE	✓	✓
DELETE	✓	✗

Sintaxis Completa

Trigger Function:

```
CREATE OR REPLACE FUNCTION nombre()  
RETURNS TRIGGER AS $$  
BEGIN  
  
    -- Tu lógica aquí  
    -- Usa NEW y OLD  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

CREATE TRIGGER:

```
CREATE TRIGGER nombre_trigger  
{ BEFORE | AFTER }  
{ INSERT | UPDATE | DELETE }  
ON nombre_tabla  
[ FOR EACH { ROW | STATEMENT } ]  
[ WHEN (condición) ]  
EXECUTE FUNCTION nombre_fn();
```

Variables de contexto dentro del trigger:

Variable	Valor	Descripción
TG_OP	'INSERT','UPDATE','DELETE'	Qué operación disparó el trigger
TG_TABLE_NAME	nombre de la tabla	Tabla donde se disparó
TG_WHEN	'BEFORE' o 'AFTER'	Momento de ejecución
TG_NARGS	número entero	Cantidad de argumentos pasados



TG_OP permite que UNA función maneje INSERT, UPDATE y DELETE con IF/ELSIF.

¿Cuándo SÍ Usar Triggers?

Si la respuesta es SÍ, un trigger es probablemente la solución correcta:



Auditoría

¿Necesitas registrar QUIÉN cambió QUÉ y CUÁNDO? El app puede olvidarlo, el trigger no.



Validaciones complejas

¿La regla requiere comparar OLD vs NEW o consultar otra tabla?



Timestamps automáticos

¿Campos como `updated_at` deben actualizarse SIEMPRE, sin excepción?



Sincronización de tablas

¿Una tabla derivada debe reflejar automáticamente cambios de otra?



Seguridad a nivel de datos

¿La BD es la última línea de defensa, sin importar qué app acceda?



Regla de oro: si CUALQUIER camino de escritura (app, script, admin) debe pasar por la misma lógica → trigger.

¿Cuándo NO Usar Triggers?

Triggers mal usados son la fuente #1 de bugs misteriosos:



Lógica de negocio compleja

Si necesita condicionales de negocio, pertenece a la capa de aplicación.



Feedback al usuario

El usuario no ve los triggers. Si necesitas informar algo, hazlo en la app.



Cadenas ($A \rightarrow B \rightarrow C$)

Debugging imposible. Si un trigger dispara otro, rediseña la arquitectura.



Cálculos pesados

El trigger vive DENTRO de la transacción. Trigger lento = app lenta para TODOS.



CHECK o FK lo resuelven

CHECK (precio > 0) es más simple, más rápido y auto-documentado.



Si en duda: ¿puede el app manejar esto? Si sí → NO uses trigger. Los triggers son garantías a nivel de datos.

5 Sistemas en Producción Real

Mismos patrones de triggers, aplicados a escala creciente:

10 users	Blog personal con CMS Timestamps automáticos	BEFORE UPDATE
40 users	Clínica dental Auditoría de expedientes médicos	AFTER INSERT/UPDATE/DELETE
100 users	E-commerce PyME Control de inventario concurrente	AFTER INSERT + FOR UPDATE
1,000 users	SaaS B2B Validación de límites por plan	BEFORE INSERT + cross-table
10,000 users	Marketplace Event sourcing + pg_notify	AFTER INSERT + pg_notify



Misma herramienta, mismo SQL. Lo que cambia es el DISEÑO y las consideraciones de performance.

Blog Personal con CMS

10 users

Timestamps automáticos — el trigger más común en producción.

Problema: Los editores olvidan actualizar 'última modificación'. La BD debe garantizarlo.

```
-- Tabla de artículos del blog
CREATE TABLE articulos (
  id SERIAL PRIMARY KEY,
  titulo VARCHAR(200), contenido TEXT, autor VARCHAR(50),
  publicado BOOLEAN DEFAULT false,
  created_at TIMESTAMP DEFAULT now(),
  updated_at TIMESTAMP DEFAULT now()
);

CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_articulos_updated
BEFORE UPDATE ON articulos
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```



Patrón REUTILIZABLE: esta misma función sirve para CUALQUIER tabla con updated_at. Solo crea otro CREATE TRIGGER.

Clínica Dental — Auditoría

40 users

Todo cambio en expedientes se registra por ley (NOM-004-SSA3-2012).

Problema: Un dentista edita un diagnóstico. Legalmente, DEBES guardar quién cambió qué y cuándo.

```
CREATE TABLE audit_expedientes (  
  id SERIAL PRIMARY KEY, tabla VARCHAR(50), accion VARCHAR(10),  
  datos_antes JSONB, datos_despues JSONB,  
  usuario VARCHAR(50) DEFAULT current_user,  
  ip_address INET DEFAULT inet_client_addr(),  
  fecha TIMESTAMP DEFAULT now()  
);  
  
CREATE OR REPLACE FUNCTION fn_audit_medico() RETURNS TRIGGER AS $$  
BEGIN  
  INSERT INTO audit_expedientes(tabla, accion, datos_antes, datos_despues)  
  VALUES(TG_TABLE_NAME, TG_OP,  
    row_to_json(OLD)::jsonb, row_to_json(NEW)::jsonb);  
  RETURN NEW;  
END; $$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_audit_expedientes  
AFTER INSERT OR UPDATE OR DELETE ON expedientes  
FOR EACH ROW EXECUTE FUNCTION fn_audit_medico();
```



AFTER porque queremos registrar el hecho ya ocurrido. `row_to_json()` convierte la fila completa a JSON.

E-commerce — Inventario

100 users

Cuando se crea un pedido, el stock se reduce automáticamente.

Problema: 3 vendedores venden el mismo producto simultáneamente. Sin control, venden más stock del disponible.

```
CREATE OR REPLACE FUNCTION fn_descontar_stock() RETURNS TRIGGER AS $$
DECLARE
    stock_actual INTEGER;
BEGIN
    SELECT stock INTO stock_actual FROM productos
    WHERE id = NEW.producto_id FOR UPDATE;
    -- bloquea fila

    IF stock_actual < NEW.cantidad THEN
        RAISE EXCEPTION
        'Stock insuficiente: % disp, % pedido',
        stock_actual, NEW.cantidad;
    END IF;

    UPDATE productos SET stock = stock - NEW.cantidad
    WHERE id = NEW.producto_id;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_descontar_stock
AFTER INSERT ON pedido_items
FOR EACH ROW EXECUTE FUNCTION fn_descontar_stock();
```



FOR UPDATE bloquea la fila durante la transacción → evita condiciones de carrera entre vendedores concurrentes.

SaaS B2B — Límites de Plan

1,000 users

No permitir que un usuario use features de un plan que no tiene.

Problema: Un usuario en plan 'Básico' intenta crear más de 5 proyectos (límite de su plan).

```
CREATE OR REPLACE FUNCTION fn_validar_limite_plan() RETURNS TRIGGER AS $$
DECLARE
    limite INT; actual INT; plan_nombre TEXT;
BEGIN
    SELECT p.max_proyectos, p.nombre INTO limite, plan_nombre
    FROM suscripciones s JOIN planes p ON s.plan_id = p.id
    WHERE s.usuario_id = NEW.owner_id AND s.activa = true;

    SELECT COUNT(*) INTO actual
    FROM proyectos WHERE owner_id = NEW.owner_id;

    IF actual >= limite THEN
        RAISE EXCEPTION
        'Plan % permite máx % proyectos. Upgrade requerido.',
        plan_nombre, limite;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_limite_proyectos
BEFORE INSERT ON proyectos
FOR EACH ROW EXECUTE FUNCTION fn_validar_limite_plan();
```



BEFORE INSERT para INTERCEPTAR antes de que se cree. RAISE EXCEPTION cancela toda la transacción.

Marketplace — Event Sourcing

10,000 users

Cada acción genera un evento y notifica a sistemas suscritos.

Problema: Al publicar un producto, se debe notificar al buscador, al feed, y al sistema de recomendaciones.

```
CREATE TABLE eventos (  
  id BIGSERIAL PRIMARY KEY, tipo VARCHAR(50),  
  entidad_id INT, payload JSONB, created_at TIMESTAMP DEFAULT now()  
);  
  
CREATE OR REPLACE FUNCTION fn_evento_producto() RETURNS TRIGGER AS $$  
BEGIN  
  -- 1) Guardar evento inmutable  
  INSERT INTO eventos(tipo, entidad_id, payload)  
  VALUES('producto.' || lower(TG_OP), NEW.id, row_to_json(NEW)::jsonb);  
  
  -- 2) Notificar listeners (async via LISTEN/NOTIFY)  
  PERFORM pg_notify('productos_channel',  
    json_build_object('op', TG_OP, 'id', NEW.id)::text);  
  
  RETURN NEW;  
END; $$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_evento_producto  
AFTER INSERT OR UPDATE ON productos  
FOR EACH ROW EXECUTE FUNCTION fn_evento_producto();
```



pg_notify() es ASYNC — no bloquea la transacción. Los microservicios escuchan con LISTEN 'productos_channel'.

Patrones por Escala: Resumen

Los triggers son los mismos. Lo que cambia es el diseño alrededor de ellos.

Escala	Patrón	Tipo	Complejidad	Consideración clave
10 users	updated_at automático	BEFORE	Baja	Reutilizable en todas las tablas
40 users	Auditoría médica	AFTER	Media	Compliance legal, JSON completo
100 users	Control de stock	AFTER	Media	FOR UPDATE para concurrencia
1K users	Límites de plan	BEFORE	Alta	Joins cross-tabla en el trigger
10K users	Event sourcing	AFTER	Alta	pg_notify async, tabla de eventos



A mayor escala: (1) más cuidado con performance, (2) más patrones async (pg_notify vs lógica inline), (3) más separación entre 'registrar el hecho' y 'actuar sobre él'.

Triggers y Transacciones

Entender esto evita el 80% de los bugs con triggers:



Un trigger se ejecuta DENTRO de la misma transacción que lo disparó.



Si el trigger falla (RAISE EXCEPTION), TODA la transacción se revierte.



Si la transacción hace ROLLBACK, los cambios del trigger se deshacen.



Un trigger NO puede hacer COMMIT ni ROLLBACK por su cuenta.



Un trigger lento hace lenta TODA la transacción del usuario.

```
BEGIN → INSERT → TRIGGER ejecuta → OK: COMMIT todo | Falla: ROLLBACK todo
```


Buenas Prácticas

1

Mantenlo simple

Un trigger = UNA responsabilidad. Si hace más de una cosa, divídelo.

2

Nombra descriptivamente

trg_audit_empleados > trigger1. Prefijo trg_ + acción + tabla.

3

Documenta el POR QUÉ

COMMENT ON TRIGGER ... IS 'Requerido por NOM-004 para auditoría'.

4

Evita triggers en cascada

Si trigger A dispara trigger B → rediseña. Debugging imposible.

5

Usa WHEN para filtrar

WHEN (NEW.status != OLD.status) — solo se dispara si cambió el status.

6

Reutiliza funciones

Una fn_audit() puede servir para 20 tablas con diferentes CREATE TRIGGER.

Anti-Patrones: Lo Que NO Hacer



Trigger que llama a otro trigger

Efecto dominó impredecible. Debugging se vuelve imposible.



Lógica de negocio compleja en trigger

Debería estar en la capa de aplicación, no escondida en la BD.



Queries pesadas dentro del trigger

Bloquea la transacción del usuario. Impacta performance de todos.



Trigger sin documentación

Nadie sabe por qué existe. Eventualmente causa bugs misteriosos.



Usar trigger cuando CHECK basta

CHECK (precio > 0) es más simple, más rápido y auto-documentado.



Si heredas una BD con triggers misteriosos: `SELECT * FROM information_schema.triggers;`

Administrar Triggers

Ver triggers de una tabla:

```
SELECT trigger_name, event_manipulation,  
       action_timing, action_statement  
FROM information_schema.triggers  
WHERE event_object_table = 'empleados';
```

Deshabilitar temporalmente:

```
ALTER TABLE empleados DISABLE TRIGGER trg_audit;
```

Rehabilitar:

```
ALTER TABLE empleados ENABLE TRIGGER trg_audit;
```

Eliminar:

```
DROP TRIGGER trg_audit ON empleados;
```



DISABLE TRIGGER ALL deshabilita todos los triggers de una tabla (útil para migraciones masivas).

FOR EACH ROW vs STATEMENT

Dos granularidades de ejecución del trigger:

FOR EACH ROW

Se ejecuta UNA VEZ por cada fila afectada

Tiene acceso a NEW y OLD

UPDATE 100 filas → se ejecuta 100 veces

Uso: auditoría, validación por fila

FOR EACH STATEMENT

Se ejecuta UNA VEZ por operación completa

NO tiene acceso a NEW ni OLD

UPDATE 100 filas → se ejecuta 1 vez

Uso: log de operaciones, notificaciones

```
-- Statement-level: se ejecuta 1 vez aunque afecte 1000 filas
CREATE TRIGGER trg_log_masivo
AFTER UPDATE ON productos
FOR EACH STATEMENT
EXECUTE FUNCTION fn_log_operacion();
```



90% de los triggers son FOR EACH ROW. Usa STATEMENT solo cuando no necesitas datos de la fila.



Ejercicio en Clase

1. Levantar PostgreSQL con Docker Compose del repositorio
2. Crear tabla productos (id, nombre, precio, stock, updated_at, is_deleted)
3. Trigger BEFORE INSERT: no permitir precio negativo ni stock < 0
4. Trigger AFTER UPDATE: registrar cambios de precio en audit_log
5. Trigger BEFORE UPDATE: auto-actualizar updated_at
6. Trigger BEFORE DELETE: soft-delete (RETURN NULL + marcar is_deleted)



github.com/RamsesCamas/BDA_Slides_And_Code

Base de Datos Avanzadas · UP Chiapas · Mtro. Ramsés Camas

Resumen: Lo Que Aprendimos Hoy

CREATE FUNCTION ... RETURNS TRIGGER

Define la lógica del trigger

CREATE TRIGGER ... ON tabla

Asocia la función a un evento y tabla

BEFORE vs AFTER

Interceptar/modificar vs registrar/notificar

NEW y OLD

Acceso a datos nuevos y anteriores

RETURN NEW / NULL

Permitir o cancelar la operación

row_to_json() + JSONB

Auditoría completa en formato flexible

FOR UPDATE

Bloqueo de fila para concurrencia

pg_notify()

Notificación async a otros sistemas

Siguiente clase: Stored Procedures + Cursores (necesitas triggers como base).